- Logging a quote to standard output can be expressed as a `Consumer`, since it has input but no output.

Listing 16.9 Implementing the business logic as functions

```
@Configuration                                                    ❶
public class QuoteFunctions {
  private static final Logger log =
    LoggerFactory.getLogger(QuoteFunctions.class);                ❷

  @Bean                                                           ❸
  Supplier<Flux<Quote>> allQuotes(QuoteService quoteService) {
    return () -> {
      log.info("Getting all quotes");
      return Flux.fromIterable(quoteService.getAllQuotes())
        .delaySequence(Duration.ofSeconds(1));                    ❹
    };
  }

  @Bean                                                           ❺
  Supplier<Quote> randomQuote(QuoteService quoteService) {
    return () -> {
      log.info("Getting random quote");
      return quoteService.getRandomQuote();
    };
  }

  @Bean                                                           ❻
  Consumer<Quote> logQuote() {
    return quote -> log.info("Quote: '{}' by {}",
      quote.content(), quote.author());
  }
}
```

❶ Functions are declared as beans in a Spring configuration class.

❷ A logger used by the functions

❸ A supplier producing all the quotes

❹ Quotes are streamed one at a time with a 1-second pause between them.

❺ A supplier producing a random quote

❻ A function logging the quote received as the input

Spring Cloud Function will automatically expose all the registered functions as HTTP endpoints when the Spring web dependencies are on the classpath.